

PRACTICAL - 8

Aim: 1. Guarded calculator: read two numbers and an operator; throw a custom `DivideByZeroException` and catch invalid-number input separately, printing a clear message each time; loop until a valid calculation succeeds and use finally to log each attempt.

Code Screenshot:

GuardedCalculator.java

```
1  import java.util.Scanner;
2
3  class DivideByZeroException extends Exception {
4      public DivideByZeroException(String message) {
5          super(message);
6      }
7  }
8
9  public class GuardedCalculator {
10     public static double calculate(double A, double B, char OP)
11         throws DivideByZeroException {
12
13         switch (OP) {
14             case '+':
15                 return A + B;
16
17             case '-':
18                 return A - B;
19
20             case '*':
21                 return A * B;
22
23             case '/':
24                 if (B == 0) {
25                     throw new DivideByZeroException("Cannot divide by zero.");
26                 }
27                 return A / B;
28         }
29         return 0;
30     }
```

```

32     public static void main(String[] args) {
33
34         Scanner sc = new Scanner(System.in);
35         boolean succeed = false;
36
37         while (!succeed) {
38
39             try {
40                 System.out.print("Enter first number: ");
41                 double A = sc.nextDouble();
42
43                 System.out.print("Enter second number: ");
44                 double B = sc.nextDouble();
45
46                 System.out.print("Enter operator (+, -, *, /): ");
47                 char OP = sc.next().charAt(0);
48
49                 double result = calculate(A, B, OP);
50                 System.out.println("Result = " + result);
51                 succeed = true;
52             }
53
54             catch (java.util.InputMismatchException e) {
55                 System.out.println("Invalid number input. Please enter numbers only.");
56                 sc.nextLine();
57             }
58             catch (DivideByZeroException e) {
59                 System.out.println("Error: " + e.getMessage());
60             }
61             finally {
62                 System.out.println("Attempt completed.");
63             }
64         }
65         sc.close();
66     }
67 }

```

Output Screenshot:

```

dhrugasavaliya@DHRUVAS-MacBook-Pro:~/Desktop/Charusat/00PJ% javac GuardedCalculator.java
dhrugasavaliya@DHRUVAS-MacBook-Pro:~/Desktop/Charusat/00PJ% java GuardedCalculator
Enter first number: 10
Enter second number: 0
Enter operator (+, -, *, /): /
Error: Cannot divide by zero.
Attempt completed.
Enter first number: 10
Enter second number: 30
Enter operator (+, -, *, /): -
Result = -20.0
Attempt completed.

```

Learning Outcome:

To understand and implement exception handling in Java using custom exceptions, try-catch-finally blocks, and loops. The program demonstrates how to create and throw a custom DivideByZeroException, handle invalid number input separately using NumberFormatException, use the finally block to log every calculation attempt, and repeatedly accept input until a valid calculation is successfully completed.

Aim : 2. Stock issue: a Warehouse where issue(item, qty) throws a custom checked `OutOfStockException` carrying the shortfall, and `InvalidQuantityException` for $qty \leq 0$. Process a list of requests, catching and reporting each failure without stopping the run.

Code Screenshot:

StockIssue.java

```

1  class OutOfStockException extends Exception {
2      int shortfall;
3
4      OutOfStockException(int shortfall) {
5          this.shortfall = shortfall;
6      }
7  }
8  class InvalidQuantityException extends Exception {
9      InvalidQuantityException(String message) {
10         super(message);
11     }
12 }
13
14 class Warehouse {
15
16     int stock = 10;
17     void issue(String item, int qty)
18         throws OutOfStockException, InvalidQuantityException {
19         if (qty <= 0) {
20             throw new InvalidQuantityException(
21                 "Invalid quantity: " + qty);
22         }
23         if (qty > stock) {
24             throw new OutOfStockException(qty - stock);
25         }
26         stock = stock - qty;
27         System.out.println("Issued " + qty + " " + item);
28         System.out.println("Remaining stock: " + stock);
29     }
30 }

```

```

32 public class StockIssue {
33
34     public static void main(String[] args) {
35
36         Warehouse w = new Warehouse();
37
38         String[] items = {"Pen", "Pen", "Pen", "Pen"};
39         int[] quantities = {3, 8, 0, 5};
40
41         for (int i = 0; i < items.length; i++) {
42
43             try {
44                 w.issue(items[i], quantities[i]);
45
46             } catch (OutOfStockException e) {
47                 System.out.println(
48                     "Out of stock. Shortfall: " + e.shortfall);
49
50             } catch (InvalidQuantityException e) {
51                 System.out.println(e.getMessage());
52             }
53
54             System.out.println();
55         }
56
57         System.out.println("All requests processed.");
58     }

```

Output Screenshot:

```

dhruvasavaliya@DHRUVAs-MacBook-Pro:~/Desktop/Charusat/00PJ% javac StockIssue.java
dhruvasavaliya@DHRUVAs-MacBook-Pro:~/Desktop/Charusat/00PJ% java StockIssue
Issued 3 Pen
Remaining stock: 7
Out of stock. Shortfall: 1
Invalid quantity: 0
Issued 5 Pen
Remaining stock: 2
All requests processed.

```

Learning Outcome:

To understand and implement custom checked exceptions in Java for handling stock-related errors. The program demonstrates how to create and throw custom `OutOfStockException` carrying the required shortfall and `InvalidQuantityException` for invalid quantities, while processing multiple stock requests. It also demonstrates exception handling that reports each failure individually without stopping the execution of the remaining requests.